



RELATÓRIO DE PROJETO

Gestão de uma biblioteca

Conceção, implementação e validação

Unidade Curricular

Unidade Curricular de exemplo

Professor Regente

Docente de exemplo

IDENTIFICAÇÃO DOS ESTUDANTES

Nome

Número

Estudante A (exemplo)

A000001

Estudante B (exemplo)

A000002

Ano letivo

2026/2027

Data de entrega

15 de janeiro de 2027

Índice

1	Introdução e objetivos	2
1.1	CrITÉrios de aceitação	2
2	Conceção e decisões	2
2.1	Separação de responsabilidades	2
2.2	Pesquisa no catálogo	2
2.3	Persistência	2
3	Implementação	2
3.1	Registo de um empréstimo	2
4	Testes e resultados	3
4.1	Interpretação e limitações	3
5	Conclusão	3
	Referências	3
A	Protocolo de teste ilustrativo	4

1 Introdução e objetivos

Este exemplo mostra como descrever um projeto de programação. O sistema proposto gere livros, leitores e empréstimos de uma pequena biblioteca. O âmbito limita-se a uma aplicação local utilizada por um operador de cada vez. O documento foi composto em Typst, seguindo a documentação oficial [1]. Pretende-se registar livros, pesquisar o catálogo por identificador e controlar empréstimos. Um empréstimo só pode ser aceite quando o exemplar está disponível. A aplicação deve conservar os dados após ser encerrada e reaberta.

Exemplo didático. O projeto e os resultados apresentados são fictícios; não constituem medições de software executado.

1.1 Critérios de aceitação

- Encontrar um livro através do seu identificador.
- Impedir dois empréstimos ativos do mesmo exemplar.
- Recuperar os registos depois de reiniciar a aplicação.
- Explicar ao utilizador por que razão uma operação foi rejeitada.

2 Conceção e decisões

2.1 Separação de responsabilidades

A interface recolhe pedidos e apresenta mensagens. A camada de domínio verifica as regras de empréstimo. A persistência lê e escreve os registos. Esta separação permite alterar a forma de armazenamento sem reescrever a interação com o utilizador.



Figura 1: Responsabilidades principais e sentido dos pedidos.

A Figura 1 representa dependências funcionais, não uma sequência temporal completa. As regras de disponibilidade pertencem ao domínio, evitando duplicá-las na interface.

2.2 Pesquisa no catálogo

Foi escolhido um mapa indexado pelo identificador do livro porque esta é a chave usada nas operações principais. Uma lista permitiria uma implementação inicial simples, mas exigiria percorrer os elementos para localizar um identificador. O mapa introduz uma estrutura adicional e não resolve, por si só, pesquisas por fragmentos do título.

2.3 Persistência

O formato JSON torna os ficheiros legíveis durante o desenvolvimento. Em contrapartida, a proposta não suporta acesso concorrente nem transações. Uma base de dados seria adequada se esses requisitos passassem a existir.

3 Implementação

3.1 Registo de um empréstimo

A operação verifica primeiro a existência do exemplar e do leitor. Depois confirma a disponibilidade. Só após todas as verificações é criado o empréstimo. O excerto seguinte resume essa regra; omite detalhes de persistência e de tratamento de erros.

```
def emprestar(exemplar, leitor):  
    if not exemplar.disponivel:  
        raise ValueError("Exemplar indisponível")  
    emprestimo = Emprestimo(exemplar, leitor)  
    exemplar.disponivel = False  
    return emprestimo
```

O interesse do excerto é a ordem das verificações. A documentação do projeto deve incluir a localização da implementação completa no repositório, sem copiar todos os ficheiros para o relatório.

4 Testes e resultados

Os cenários da Tabela 1 ilustram o tipo de evidência que um relatório deve apresentar. Num projeto real, a coluna de observações deve resultar da execução dos testes e indicar versão, ambiente e dados utilizados.

Cenário	Resultado esperado	Observação
Exemplar disponível	Empréstimo aceite	Aceite
Segundo empréstimo	Operação rejeitada	Rejeitada
Leitor inexistente	Mensagem de erro	Apresentada
Reabrir a aplicação	Registos preservados	Preservados

Tabela 1: Cenários e observações fictícias para fins didáticos.

4.1 Interpretação e limitações

As observações ilustradas são compatíveis com os critérios de aceitação definidos, mas quatro cenários não demonstram ausência de defeitos. Faltaria testar ficheiros corrompidos, falhas durante a escrita e entradas inesperadas. Não foram realizados ensaios de desempenho nem testes com vários operadores.

5 Conclusão

A solução proposta cobre o registo de livros, a pesquisa por identificador e o controlo de empréstimos no âmbito de uma aplicação local. A separação entre interface, domínio e persistência torna explícitas as responsabilidades de cada componente.

Os cenários apresentados exemplificam como relacionar requisitos e evidência. Num relatório real, a conclusão deve apoiar-se em resultados obtidos. O acesso concorrente e a recuperação de falhas de escrita permanecem fora do âmbito desta proposta e constituem prioridades para uma evolução futura.

Referências

- [1] Typst, «Typst Documentation». Acedido: 22 de setembro de 2026. [Em linha]. Disponível em: <https://typst.app/docs/>

A Protocolo de teste ilustrativo

1. Criar um catálogo de teste com um exemplar disponível.
2. Registrar um leitor válido.
3. Solicitar um empréstimo e verificar o estado do exemplar.
4. Repetir o pedido e confirmar que não foi criado um segundo empréstimo.
5. Guardar, encerrar e reabrir; comparar os registos.

Num trabalho real, juntar os comandos, os dados de entrada e os resultados necessários para reproduzir o teste.